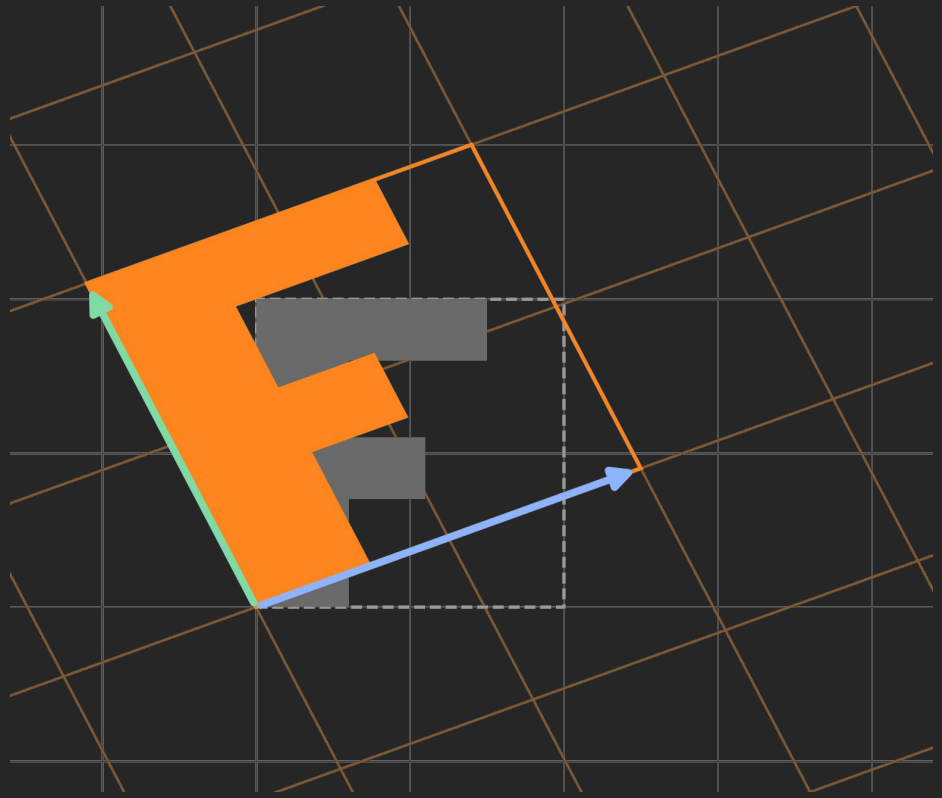
LEAD UNIVERSITY

BCD3103 · CLASE 7 · INGENIERÍA EN CIENCIA DE DATOS

Matrices en acción: transformar, medir y acelerar



Qué le hace una matriz al espacio, cuándo pierde información y cómo se calcula rápido a gran escala.

Cada tema: explicación · ejemplos resueltos · práctica · solución

RUTA DE LA CLASE

De la geometría al rendimiento

Primero entendemos qué hace una matriz con el espacio; después, cuánto cuesta calcularla.

BLOQUE 1 · Transformaciones y espacios

- **Matrices como transformaciones**
  Escala, rota, refleja e inclina el plano
- **Espacio columna y espacio nulo**
  Qué produce A y qué destruye
- **Rango y dependencia lineal**
  Cuánta información independiente hay
- **Sobredeterminados y colinealidad**
  Más ecuaciones que incógnitas

BLOQUE 2 · Cómputo matricial y rendimiento

- **Vectorización vs ciclos**
  Operar sobre arreglos completos
- **Matrices densas vs dispersas**
  Guardar solo lo que no es cero
- **Complejidad computacional**
  Contar operaciones antes de correr
- **Benchmarking y eficiencia**
  Medir bien y elegir la precisión

Cómo está armada cada sección

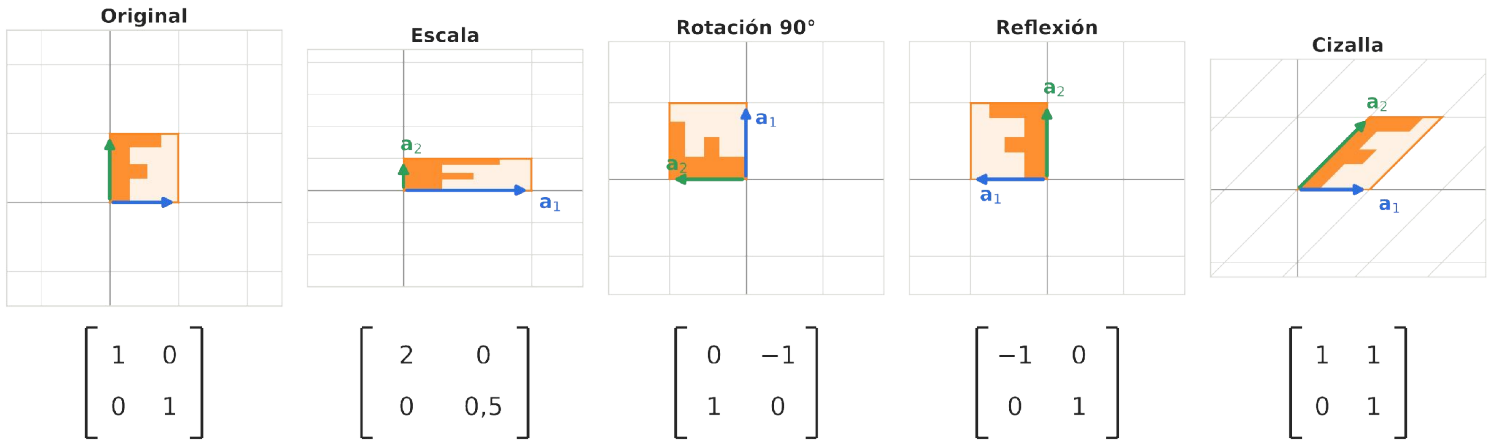
1. **Explicación** con dibujo
2. **Ejemplos** paso a paso
3. **Práctica** usted resuelve
4. **Solución** autocorrección

→ **Para recordar:** Al terminar, deberían poder dibujar qué hace una matriz y estimar cuánto cuesta calcularla.

Una matriz es una función que mueve el espacio

Ax toma un vector y lo lleva a otro: la matriz A es la regla de movimiento.

En pocas palabras: Multiplicar por A transforma todo el plano a la vez. Para saber qué hace A, basta ver a dónde caen e_1 y e_2: son sus columnas.



$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 2 & 0 \\ 0 & 0{,}5 \end{bmatrix} \quad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \quad \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Las columnas lo dicen todo

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$A \cdot e_1$ = 1.ª columna, $A \cdot e_2$ = 2.ª columna

Escala, rota, refleja o inclina

Rectas siguen siendo rectas
y el origen no se mueve

Producto = una tras otra
AB: aplicar B y luego A

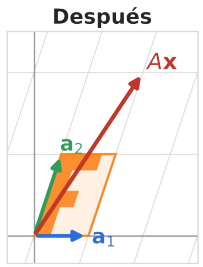
→ **Para recordar:** si dos matrices mandan e_1 y e_2 a los mismos lugares, son la misma transformación.

Tres transformaciones paso a paso

Calculamos con la regla Ax y comprobamos con la geometría.

1 Aplicar una matriz a un vector





$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

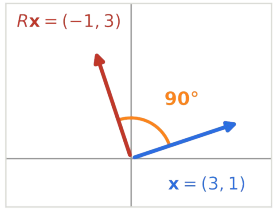
Fila 1: $2 \cdot 1 + 1 \cdot 2 = 4$

Fila 2: $0 \cdot 1 + 3 \cdot 2 = 6$

$e_1 \to (2, 0)$ y $e_2 \to (1, 3)$

Resultado: Ax = (4, 6)

2 Rotar 90° a la izquierda



$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$$

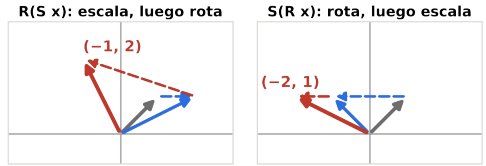
R tiene columnas (0, 1) y (−1, 0)

$Rx = (0 \cdot 3 - 1 \cdot 1, 1 \cdot 3 + 0 \cdot 1)$

Longitud: $\|x\| = \|Rx\| = \sqrt{10}$

Resultado: Rx = (−1, 3)

3 El orden importa



$$RS = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 2 & 0 \end{bmatrix}$$

$$SR = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 1 & 0 \end{bmatrix}$$

RS: primero escala, luego rota

En x = (1, 1):

$RS \to (-1, 2)$ y $SR \to (-2, 1)$

Resultado: RS ≠ SR

Práctica: cómo se mueven los vectores

Resuelva sin ver la solución. Pista: ¿a dónde caen e_1 y e_2?

Problema 1

Una matriz curiosa

Sea A la matriz de la figura.

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \qquad x = \begin{bmatrix} 3 \\ 5 \end{bmatrix}$$

a) Calcule Ax.

b) ¿Qué le hace A a cualquier vector (x, y)?

Problema 2

Construir la matriz

Escriba la matriz que estira x por 3 y cambia el signo de y. Luego aplíquela a x.

$$A = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix} \qquad x = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

a) Complete la matriz (columnas = imágenes de e_1 y e_2).

b) Calcule Ax.

Problema 3

Una cizalla

Aplique A a los vértices (0,0), (1,0), (1,1) y (0,1).

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

a) Calcule las cuatro imágenes.

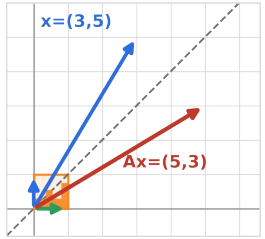
b) Dibuje la figura. ¿Qué forma tiene?

→ **Pista:** dibuje e_1 y e_2 y observe a dónde van; las columnas de A son esas imágenes.

Solución paso a paso

Cada resultado se lee en las columnas de la matriz.

1 Intercambiar coordenadas



$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$$

Fila 1: $0 \cdot 3 + 1 \cdot 5 = 5$

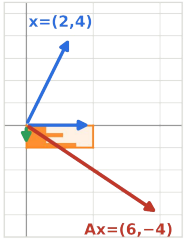
Fila 2: $1 \cdot 3 + 0 \cdot 5 = 3$

$e_1 \to (0, 1)$, $e_2 \to (1, 0)$

Es una reflexión sobre $y = x$

Resultado: **Ax = (5, 3)**

2 Estirar x y reflejar y



$$\begin{bmatrix} 3 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ -4 \end{bmatrix}$$

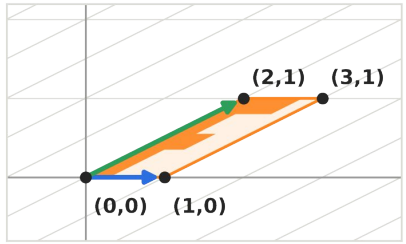
$e_1 \to (3, 0)$ y $e_2 \to (0, -1)$

Por eso $A = [[3, 0], [0, -1]]$

$3 \cdot 2 = 6$ y $(-1) \cdot 4 = -4$

Resultado: **Ax = (6, −4)**

3 Cizalla del cuadrado unitario



$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 3 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Vértices como columnas: una sola multiplicación

Imágenes: (0,0), (1,0), (3,1), (2,1)

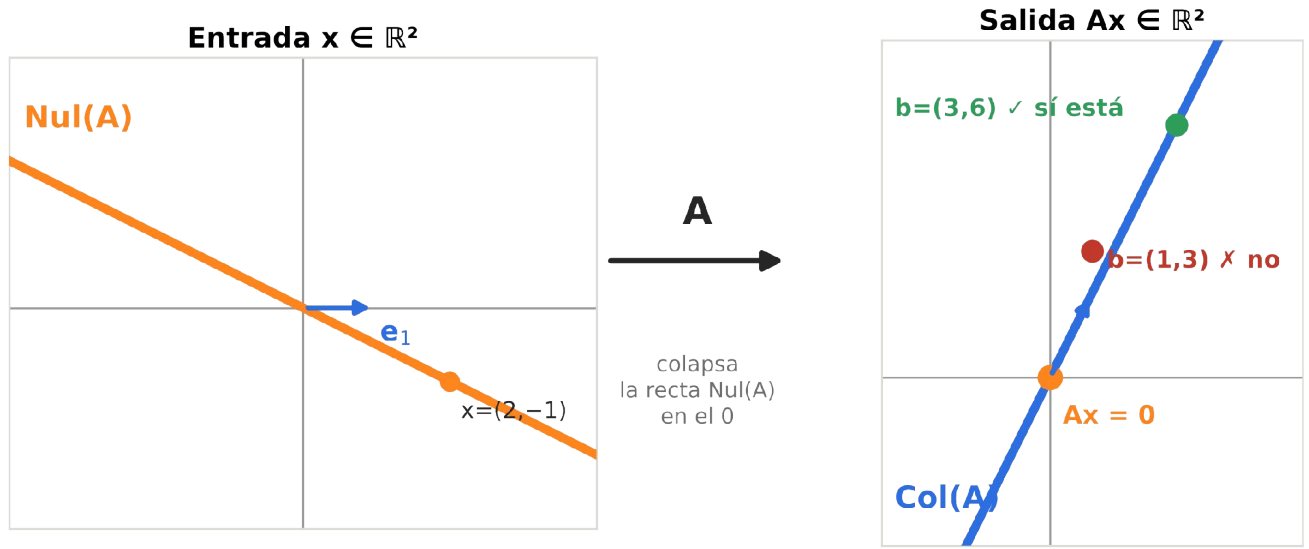
La base queda fija y el techo se corre 2

Resultado: **Paralelogramo**

Qué produce A y qué destruye

Dos subespacios cuentan toda la historia de una matriz.

En pocas palabras: Col(A) son todas las salidas posibles de A (combinaciones de sus columnas). Nul(A) son las entradas que A convierte en 0.



Col(A): lo que A puede producir

Todas las combinaciones lineales de las columnas. Vive en el espacio de llegada.

Nul(A): lo que A anula

Los x con Ax = 0. Vive en el espacio de salida; si solo tiene a 0, A no pierde información.

Para recordar: Ax = b tiene solución exactamente cuando b está en Col(A).

Tres matrices, dos subespacios

Buscamos qué salidas produce A y qué entradas anula.

1 Columnas proporcionales

$c_1 = (1, 2)$ **$c_2 = 2 \cdot c_1$**

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \cdot \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Todas las columnas están en una recta

Col(A) = gen{(1, 2)}

Nul: $x + 2y = 0 \rightarrow t(-2, 1)$

Resultado: **Col: recta · Nul: recta**

2 Dos columnas independientes

rango 2 **nulidad 1**

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

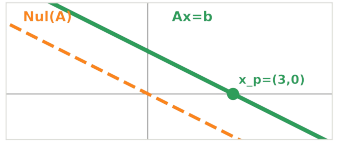
$c_1 = e_1$ y $c_2 = e_2$ generan todo $\mathbb{R}^2$

Nul: $x_1 = -x_3$ y $x_2 = -x_3$

3 columnas = rango 2 + nulidad 1

Resultado: **Col = $\mathbb{R}^2$ · rango 2**

3 Resolver Ax = b



$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$$

$b = (3, 6) = 3 \cdot c_1$ está en Col(A)

Particular (3, 0) + t(−2, 1)

Resultado: **x = (3, 0) + t(−2, 1)**

→ **Para recordar:** Col(A) = lo que A produce · Nul(A) = lo que A anula · Ax = b: solución particular + Nul(A).

Práctica: describir Col(A) y Nul(A)

Resuelva sin ver la solución. Pista: busque relaciones entre columnas.

Problema 1

Sistema con columnas repetidas

$$A = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \qquad b = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$$

a) Describa Col(A) y Nul(A).

b) ¿Ax = b tiene solución?

c) Escriba la solución general.

Problema 2

Una matriz ancha

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \end{bmatrix}$$

a) Halle Nul(A) con una base.

b) ¿Cuál es el rango de A?

c) ¿Es Col(A) = $\mathbb{R}^2$?

Problema 3

Más filas que columnas

Para A de 3×2:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

a) ¿Qué forma tiene Col(A) en $\mathbb{R}^3$?

b) ¿Ax = (1,2,3) es consistente?

c) ¿Y con (1,2,4)?

→ **Pista:** busque relaciones entre columnas: dicen el rango y quién está en Nul(A).

Solución paso a paso

Verificamos siempre multiplicando A por el vector hallado.

1 Columnas opuestas

$c_1 = (1, 2)$ **$c_2 = -c_1$**

$$\begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Col(A) = gen{(1, 2)}

Nul: $x - y = 0 \rightarrow t(1, 1)$

$b = 3 \cdot c_1 \rightarrow x = (3, 0) + t(1, 1)$

Resultado: **Nul(A) = gen{(1, 1)}**

2 Matriz ancha de rango 2

rango 2 **nulidad 1**

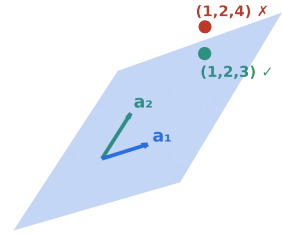
$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$x_2 = -x_3$ y $x_1 = -2x_2 - 3x_3 = -x_3$

Dos pivotes → rango 2 → Col(A) = $\mathbb{R}^2$

Resultado: **Nul = t(−1, −1, 1)**

3 El plano z = x + y



Col(A) = {(x, y, x + y)}: un plano

(1, 2, 3): 3 = 1 + 2 → sí está

(1, 2, 4): 4 ≠ 1 + 2 → no está

Resultado: **Plano z = x + y**

→ **Verifique:** el vector hallado debe cumplir $A \cdot x = 0$.

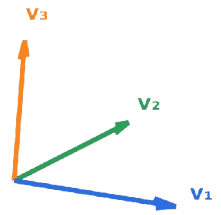
RANGO · EXPLICACIÓN

Rango: cuánta información independiente hay

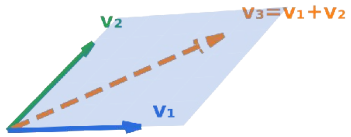
El rango es la dimensión de Col(A): cuántas columnas realmente aportan algo nuevo.

En pocas palabras: Una columna que se obtiene combinando otras no agrega dirección nueva. Rango = columnas independientes = pivotes.

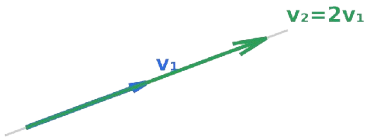
Rango 3: llenan el espacio



Rango 2: caen en un plano



Rango 1: caen en una recta



Teorema rango–nulidad



A de 4×6: 6 columnas

rango = 4 | **nulidad = 2**

rango + nulidad = n.º de columnas

Rango = pivotes
tras escalonar la matriz

Columna dependiente
es combinación de las otras

Rango completo
sin pérdida de información

→ **Para recordar:** rango bajo significa información repetida; nulidad > 0 significa que A pierde información.

Calcular el rango de tres formas

Por relaciones entre filas, entre columnas y por eliminación.

1 Una fila es múltiplo de otra

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$$

$F_2 = 2 \cdot F_1 \rightarrow$ fila de ceros al eliminar

$c_3 = c_1 + c_2 \rightarrow$ columna sobrante

Nul: $t(1, 1, -1)$

Resultado: rango 2 · nulidad 1

2 ¿Son dependientes v_1, v_2, v_3?

$v_1 = (1,2,1)$ $v_2 = (0,1,1)$

$v_3 = (1,3,2) = v_1 + v_2$

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & 3 \\ 1 & 1 & 2 \end{bmatrix}$$

Columnas v_1, v_2, v_3 de la matriz

$v_3 - v_1 - v_2 = 0$

Resultado: dependientes · rango 2

3 Contar pivotes



3 pivotes = 3 columnas independientes

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 3 \\ 1 & 3 & 3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

$F_2 - 2F_1$ y $F_3 - F_1$

luego $F_3 - F_2$

Tres pivotes: no se pierde nada

Resultado: rango 3 · invertible

→ **Para recordar:** tres caminos al mismo número: filas, columnas o pivotes.

Práctica: rango, nulidad y dependencia

Resuelva sin ver la solución.

Problema 1

Filas proporcionales

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}$$

a) Calcule el rango de A.
b) Halle Nul(A).
c) ¿Son independientes sus columnas?

Problema 2

Una columna sobrante

$$B = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 1 & 1 & 5 \end{bmatrix}$$

a) Halle la relación entre columnas.
b) Calcule el rango de B.

Problema 3

Solo conocemos las dimensiones

A es de 4×6 y tiene rango 3.



a) ¿Cuál es la nulidad?
b) ¿Cuál es el mayor rango posible?
c) ¿Ax = b siempre tiene solución?

→ **Pista:** escalone y cuente pivotes; después use rango + nulidad = n.

Solución paso a paso

El rango sale de contar pivotes; la nulidad, de n − rango.

1 Rango 1

$F_2 = 3 \cdot F_1$ y $c_2 = 2 \cdot c_1$

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix}$$

Una sola fila independiente

Nul: x + 2y = 0 → t(−2, 1)

Columnas dependientes

Resultado: rango 1 · nulidad 1

2 Rango 2

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 1 & 1 & 5 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix}$$

$F_3 - F_1 - F_2 = (0, 0, 0)$

Columnas: $c_3 = 2c_1 + 3c_2$

Dos pivotes → rango 2

Resultado: rango 2 · $c_3 = 2c_1 + 3c_2$

3 Solo con dimensiones



3 + 3 = 6 columnas

Nulidad = 6 − 3 = 3

Rango máximo = min(4, 6) = 4

Col(A) tiene dimensión 3 < 4: hay b sin solución

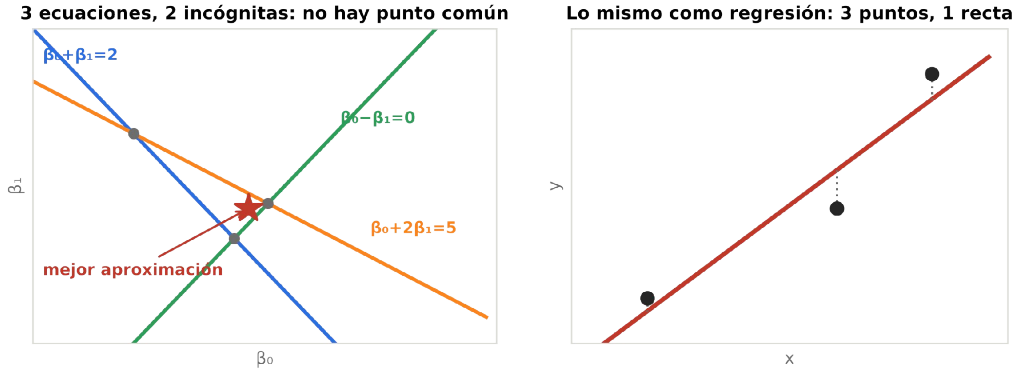
Resultado: nulidad 3 · rango ≤ 4

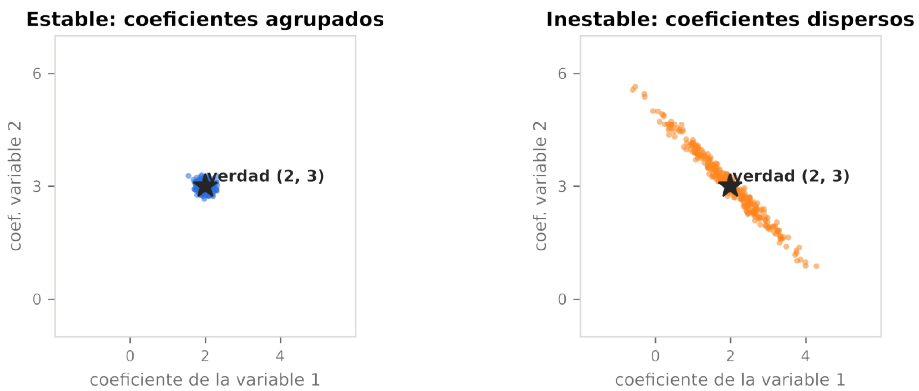
→ **Verifique:** compruebe siempre: rango + nulidad = número de columnas.

Sobredeterminados y colinealidad

Resolvemos $X^TX\beta = X^Ty$, pero si las columnas se parecen el resultado se vuelve inestable.

En pocas palabras: si dos variables dicen casi lo mismo, el modelo no sabe cuánto atribuir a cada una: los coeficientes oscilan.





×7
más incertidumbre en β (0,12 → 0,88)

VIF ≈ 50
con r = 0,99 entre dos variables

cond(X)²
cond(X^TX) = cond(X)²: peor condicionada

Para recordar: VIF = 1/(1 − r²): con r cerca de 1 se dispara. Más ecuaciones que incógnitas se resuelve por mínimos cuadrados.

Ajustar, detectar y medir

Tres situaciones donde X tiene más filas que columnas.

1 Ecuaciones normales

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 5 \end{bmatrix}$$

3 ecuaciones, 2 incógnitas: sin solución exacta

Multiplicamos por X^T:

Resultado: β ≈ (1,29; 1,57)

2 Columnas colineales

$x_1 = (1, 2, 3)$ $x_2 = 2 \cdot x_1$

$$X^TX = \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}$$

$x_2 = 2 \cdot x_1$: variables redundantes

det = 14·56 − 28·28 = 0

No existe $(X^TX)^{-1}$

Resultado: X^TX no invertible

3 Número de condición

cond(X) = 134

cond(X^TX) = 17 950

$134^2 = 17\,956 \approx 17\,950$

Cada ecuación normal agranda el error ×134

Se pierden unas 4 cifras decimales

Resultado: cond empeora ×134

→ **Para recordar:** colinealidad ⇒ X^TX casi singular ⇒ coeficientes inestables.

Práctica: ajustar y detectar colinealidad

Resuelva sin ver la solución.

Problema 1

Tres puntos, una recta

Ajuste $y = \beta_0 + \beta_1 x$ a (0,1), (1,2), (2,2).

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \qquad y = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$

a) Calcule X^TX y X^Ty.

b) Resuelva para β.

Problema 2

¿Se puede ajustar?

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

a) ¿Qué relación hay entre columnas?

b) Dé el rango de X.

c) ¿X^TX es invertible?

Problema 3

Metros y pies cuadrados

Un modelo de precios usa el área en m² y en pies² (1 m² ≈ 10,76 pies²).

área m² ↔ **área pies²**

a) ¿Qué problema aparece?

b) ¿Qué pasa con el VIF?

c) ¿Qué haría usted?

→ **Pista:** arme X^TX y X^Ty; con columnas proporcionales, X^TX no se puede invertir.

Solución paso a paso

Calculamos, comprobamos y decidimos qué hacer con la colinealidad.

1 Recta de mínimos cuadrados

$$\begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix} \cdot \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$X^TX = [[3, 3], [3, 5]]$, $X^Ty = (5, 6)$

$\det = 15 - 9 = 6$

$\beta_0 = (25 - 18)/6 = 7/6$

$\beta_1 = (18 - 15)/6 = 1/2$

Resultado: $\hat{y} = 7/6 + x/2$

2 Rango deficiente

rango 2 **falta 1**

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

$c_2 = 2 \cdot c_1 \rightarrow$ columnas dependientes

rango $2 < 3 \rightarrow X^TX$ singular

Quite una variable o use Ridge

Resultado: X^TX no es invertible

3 Variables duplicadas

pies² = 10,76 · m² → r = 1

$VIF = 1/(1 - r^2) \rightarrow \infty$

β inestables: el modelo no reparte el efecto

Conserve una sola área, o use Ridge o PCA

Resultado: Conserve una variable

→ **Verifique:** si el rango es menor que el número de columnas, quite variables o regularice.

VECTORIZACIÓN · EXPLICACIÓN

Vectorizar: operar el arreglo completo

Se escribe menos código y corre cientos de veces más rápido.

En pocas palabras: un ciclo for procesa un número por vuelta; la versión vectorizada entrega todo el arreglo a código compilado que usa instrucciones SIMD.

Tiempo relativo, producto punto de 10^6 (a @ b = 1)



Qué pasa por dentro

Ciclo

1 multiplicación por vuelta + intérprete

Vectorizado

varias a la vez (SIMD), sin intérprete

a @ b es ≈ 400 veces más rápido que el ciclo sobre arreglo

Sin intérprete por elemento
el ciclo corre en C

Instrucciones SIMD
varios números por ciclo

Menos código
una línea en vez de un ciclo

Para recordar: si hay un for sobre elementos de un arreglo, casi siempre hay una versión vectorizada.

Tres cálculos, sin ciclos

Mismo resultado, mucho menos tiempo (valores medidos en el equipo de preparación).

1 Producto punto

```
# ciclo
s = 0.0
for i in range(n):
    s += a[i] * b[i]
# vectorizado
s = a @ b
```

$n = 10^6$ elementos

Ciclo sobre lista: 41 ms

a @ b: 0,6 ms

Resultado: ≈ 70 veces más rápido

2 Producto de matrices

```
# 3 ciclos anidados
for i in range(m):
  for j in range(m):
    for k in range(m):
      C[i][j] += A[i][k]*B[k][j]
# vectorizado
C = A @ B
```

m = 200: 385 ms vs 0,37 ms

Resultado: ≈ 1 000 veces más rápido

3 Estandarizar columnas

X: n × 3 − **μ: 3**

```
mu = X.mean(axis=0)  # (3,)
sd = X.std(axis=0)   # (3,)
Z = (X - mu) / sd    # (n, 3)
```

μ se repite en cada fila (broadcasting)

Resultado: Z con media 0 y desv. 1

→ **Para recordar:** vectorizar no cambia el resultado: cambia quién ejecuta el ciclo.

Práctica: reemplace el ciclo

Escriba la expresión vectorizada. Sin ciclos for.

Problema 1

Distancia a un punto

X es n×3 (n puntos) y p = (1, 2, 3). Calcule la distancia de cada fila a p.

```
p = np.array([1, 2, 3])
d = ...   # forma (n,)
```

a) Escriba la expresión.
b) ¿Qué hace axis=1?

Problema 2

Normalizar filas

U es m×4. Divida cada fila por su norma para que mida 1.

```
U_unit = ...   # forma (m, 4)
```

a) Escriba la expresión.
b) ¿Para qué sirve keepdims=True?

Problema 3

Contar positivos

x tiene 10^6 números. Cuente cuántos son positivos.

```
cuenta = ...   # un entero
```

a) Escriba la expresión.
b) ¿Qué valor numérico tiene True?

→ **Pista:** una comparación como x > 0 devuelve un arreglo booleano; sumarlo cuenta los True.

Solución paso a paso

Cada expresión se comprueba mirando la forma del resultado.

1 Distancia a p

```
sq = (X - p) ** 2
d = np.sqrt(sq.sum(axis=1))
```

X − p resta p a cada fila

axis=1 suma por filas → (n,)

Resultado: d.shape = (n,)

2 Filas de norma 1

```
nrm = np.linalg.norm(
    U, axis=1, keepdims=True)
U_unit = U / nrm
```

keepdims=True deja nrm en (m, 1)

Cada fila se divide por su norma

Resultado: cada fila con norma 1

3 Contar con una máscara

```
mask = x > 0       # booleanos
cuenta = mask.sum()
```

True cuenta como 1, False como 0

Una sola pasada, sin ciclo Python

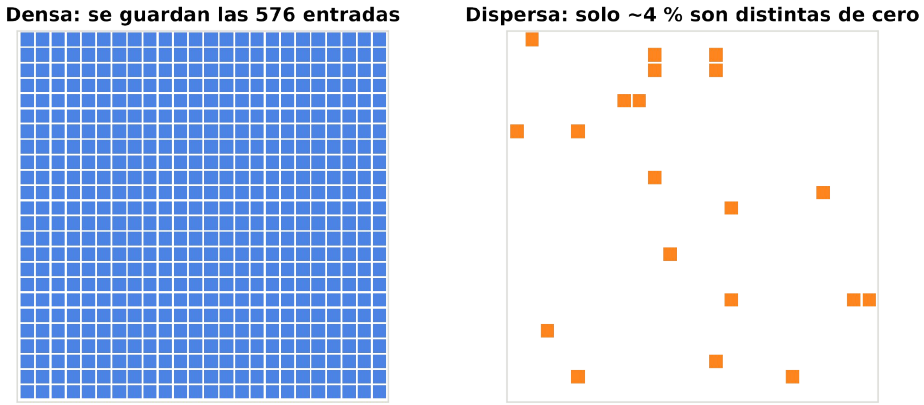
Resultado: (x > 0).sum()

→ **Verifique:** la forma del resultado debe ser la esperada: (n,), (m, 4) o un número.

Guardar solo lo que no es cero

Una matriz dispersa tiene casi todos sus elementos en cero; no tiene sentido almacenarlos.

En pocas palabras: en formato CSR se guardan solo los valores no nulos (data), su columna (indices) y dónde empieza cada fila (indptr).



Matriz 4 × 4

f0	5	0	0	1
f1	0	0	2	0
f2	0	0	0	0
f3	3	0	0	4

data (los valores ≠ 0)

5	1	2	3	4

indices (columna de cada valor)

0	3	2	0	3

indptr (dónde empieza cada fila)

0	2	3	3	5

Fila 0 ocupa las posiciones indptr[0]:indptr[1] = 0:2 → valores 5 y 1

Densa: guarda todo
n×m números, haya o no ceros

CSR: tres arreglos
data, indices, indptr

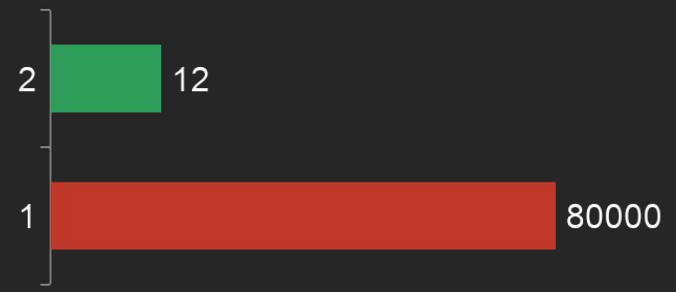
Ahorro enorme
si hay pocos no ceros

Para recordar: la dispersión importa: memoria y operaciones dependen de los no ceros, no del tamaño total.

El ahorro en números

Matriz de 100 000 × 100 000 con 10^6 entradas no nulas (10 por fila).

1 Memoria (MB, escala log)



Densa: $10^{10} \times 8$ B = 80 GB

CSR: $10^6 \times 12$ B ≈ 12 MB

Resultado: ≈ 6 450 veces menos

2 Convertir a CSR

```
from scipy.sparse import \
    csr_matrix
A = csr_matrix(M)
A.nnz          # no ceros
y = A @ x      # matvec
```

Solo recorre los no ceros

El resultado y es un arreglo normal

Resultado: resultado idéntico

3 Costo de A·x

densa: $2n^2 = 2 \cdot 10^{10}$

CSR: $2 \cdot nnz = 2 \cdot 10^6$

$n = 10^5$ y $nnz = 10^6$

Cociente: $2 \cdot 10^{10} / 2 \cdot 10^6 = 10^4$

Resultado: 10^4 veces menos

→ **Para recordar:** CSR gana cuando la matriz es muy dispersa; con mucha densidad conviene la densa.

Práctica: leer y estimar con CSR

Resuelva sin ver la solución.

Problema 1

Pasar a CSR

$$A = \begin{bmatrix} 0 & 7 & 0 & 0 \\ 2 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 \end{bmatrix}$$

a) Escriba data, indices e indptr.

b) ¿Cuántos no ceros hay?

Problema 2

Matriz TF-IDF

50 000 documentos × 20 000 palabras; solo 0,2 % de entradas no nulas.

a) Memoria densa (float64).

b) Memoria CSR aprox. (12 B por no cero).

c) ¿Cuántas veces menos?

Problema 3

Reconstruir la matriz

Matriz de 3×3 con:

```
data    = [3, 1, 2]
indices = [2, 0, 1]
indptr  = [0, 1, 1, 3]
```

a) Escriba la matriz A.

b) ¿Qué fila está vacía?

→ **Pista:** indptr[i] a indptr[i+1] marca los valores de la fila i dentro de data e indices.

Solución paso a paso

Primero la estructura, después la memoria.

1 Pasar a CSR

```
data    = [7, 2, 3, 4]
indices = [1, 0, 3, 2]
indptr  = [0, 1, 3, 3, 4]
```

Fila 0: 7 · Fila 1: 2, 3

Fila 2: vacía · Fila 3: 4

Resultado: 4 no ceros

2 Memoria TF-IDF

densa: 8 GB

CSR: ≈ 24 MB

Densa: $5 \cdot 10^4 \cdot 2 \cdot 10^4 \cdot 8$ B = 8 GB

No ceros: $0{,}002 \cdot 10^9 = 2 \cdot 10^6 \rightarrow$ 24 MB

Resultado: ≈ 330 veces menos

3 Reconstruir A

[[0,0,3],[0,0,0],[1,2,0]]

Fila 0: columna 2 → 3

Fila 1: vacía (indptr 1 = 1)

Fila 2: col 0 → 1, col 1 → 2

Resultado: fila 1 vacía

→ **Verifique:** la longitud de data e indices es el número de no ceros; indptr tiene una más que las filas.

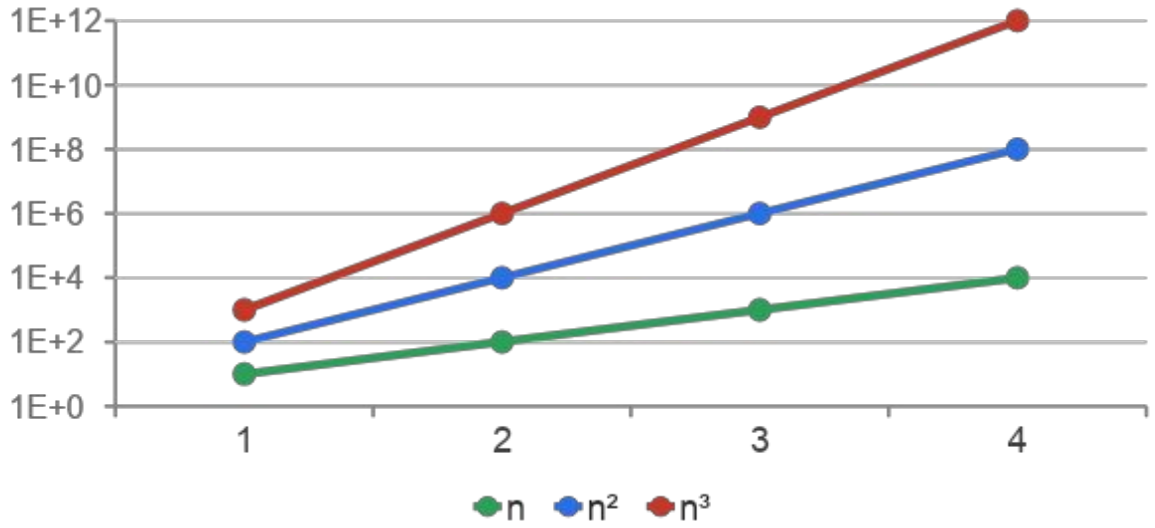
COMPLEJIDAD · EXPLICACIÓN

Contar operaciones antes de correr

La complejidad dice cómo crece el trabajo cuando crece n.

En pocas palabras: si n se multiplica por k, un algoritmo de orden n^3 hace k^3 veces más trabajo. Duplicar n cuesta 8 veces; multiplicar por 10, mil veces.

Operaciones según n (escala logarítmica)



Costo aproximado en flops

Operación	Costo
Producto punto (n)	**2n**
Matriz × vector (n×n)	**$2n^2$**
Matriz × matriz (n×n)	**$2n^3$**
Resolver Ax = b	**$\approx (2/3)n^3$**

Un flop = una suma o una multiplicación en punto flotante.

→ **Para recordar:** antes de ejecutar, estime los flops; después, mida para confirmar.

Estimar costos con papel y lápiz

Contamos operaciones y predecimos el tiempo.

1 Flops de un producto

1000×500 · 500×200

Costo = $2\cdot m\cdot k\cdot n$

$2 \cdot 1000 \cdot 500 \cdot 200$

$= 2 \times 10^8$ operaciones

Resultado: 2×10^8 flops

2 Predecir el tiempo

$n \to 4n$ → $\times 4^3 = 64$

Producto de matrices: tiempo $\propto n^3$

Hoy: n = 1 000 tarda 2 s

Con 4 000: 2 s × 64

Resultado: ≈ 128 s

3 El orden de los productos

$(AB)x$: $2n^3 = 2\cdot10^9$ flops

$A(Bx)$: $4n^2 = 4\cdot10^6$ flops

n = 1 000; A, B cuadradas, x vector

Medido: 32 ms vs 0,6 ms

Resultado: A(Bx): 500× menos flops

→ **Para recordar:** multiplique de derecha a izquierda cuando termine en un vector.

Práctica: contar flops y predecir tiempos

Resuelva sin ver la solución.

Problema 1

Matriz por vector

Una matriz de 3 000 × 4 000 se multiplica por un vector.

a) ¿Cuántos flops cuesta (2·m·n)?

b) Exprese el resultado con potencias de 10.

Problema 2

Predecir el tiempo

Una operación de orden n^2 tarda 5 ms con n = 1 000.

a) ¿Cuánto tardará con n = 3 000?

b) ¿Y con n = 10 000?

Problema 3

¿Qué orden conviene?

A y B son de 5 000 × 5 000 y x es un vector.

a) Flops de (AB)x.

b) Flops de A(Bx).

c) ¿Cuántas veces menos?

→ **Pista:** si n se multiplica por k, el trabajo de orden n^p se multiplica por k^p.

Solución paso a paso

Potencias de 10 en lugar de calculadora.

1 Matriz por vector

$2{,}4 \times 10^7$ flops

Costo = $2 \cdot m \cdot n$

$2 \cdot 3\,000 \cdot 4\,000$

$= 2 \cdot 12 \cdot 10^6$

Resultado: $2{,}4 \times 10^7$ flops

2 Escalar un costo n^2

$n \times 3 \rightarrow \times 9$

$n = 3\,000$: $3^2 = 9 \rightarrow 5$ ms $\cdot$ 9 = 45 ms

$n = 10\,000$: $10^2 = 100 \rightarrow 500$ ms

Resultado: 45 ms y 500 ms

3 (AB)x contra A(Bx)

$(AB)x$: $2n^3 = 2{,}5 \cdot 10^{11}$

$A(Bx)$: $4n^2 = 10^8$

$2{,}5 \cdot 10^{11} / 10^8 = 2\,500$

Resultado: ≈ 2 500 veces menos

→ **Verifique:** la razón de costos sale de dividir las potencias, no de calcular todo.

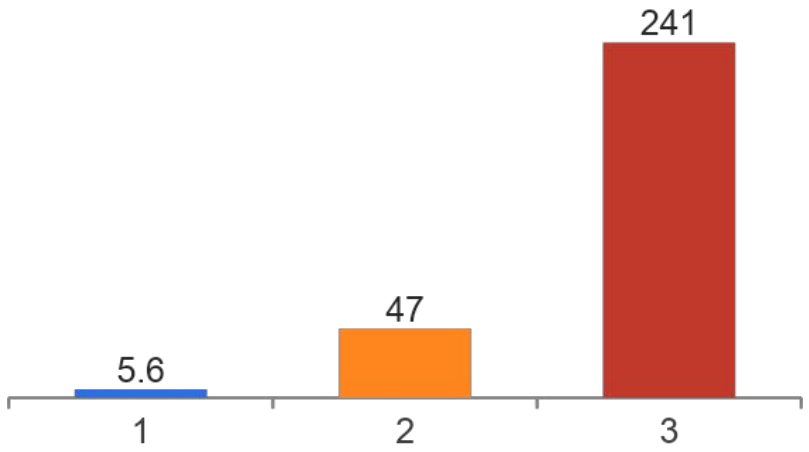
BENCHMARKING · EXPLICACIÓN

Medir bien y elegir la precisión

Un buen benchmark es repetible; una buena precisión es la necesaria, no la máxima.

En pocas palabras: calentar, repetir y reportar la mediana. Y elegir float32 o float64 según cuántas cifras necesita realmente el problema.

Producto A @ B (ms)



Receta de un benchmark

1. **Calentar** una ejecución de prueba
2. **Repetir** 5 a 10 veces
3. **Mediana** no la media
4. **Fijar** tamaño y semilla

float32 vs float64

float32 · 4 bytes

$\varepsilon \approx 1{,}2 \cdot 10^{-7}$ (7 cifras)

float64 · 8 bytes

$\varepsilon \approx 2{,}2 \cdot 10^{-16}$ (16 cifras)

float32: mitad de memoria

→ **Para recordar:** mediana de varias repeticiones; y float32 solo si 7 cifras bastan.

Medir, resumir y elegir

Tres decisiones que cambian la conclusión.

1 ¿Cuánto más rápido?

ciclos: 385 ms

A @ B: 0,37 ms

Razón = 385 / 0,37

m = 200, matrices 200 × 200

Resultado: ≈ 1 000 veces

2 Media o mediana

9,9 | 10,1 | 10,3 | 10,1 | 25,3

Cinco tiempos en ms

Media = 13,14 (la arrastra 25,3)

Mediana = 10,1

Resultado: reportar la mediana

3 Elegir precisión

```
np.float32(16777216) + 1
# → 16777216.0   (pierde el 1)
```

float32: $\varepsilon \approx 1{,}2 \cdot 10^{-7}$

float64: $\varepsilon \approx 2{,}2 \cdot 10^{-16}$

Resultado: float32 pierde el +1

→ **Para recordar:** la media castiga los valores atípicos; la mediana los ignora.

BENCHMARKING · PRÁCTICA

Práctica: interpretar mediciones

Resuelva sin ver la solución.

Problema 1

Aceleración

Un cálculo tarda 12 s con ciclos y 0,03 s vectorizado.

a) ¿Cuántas veces más rápido es?

b) ¿Cómo lo mediría de forma confiable?

Problema 2

Mediciones con ruido

Cinco tiempos (ms): 8,0 · 8,2 · 8,4 · 8,2 · 19,3.

a) Calcule la media.

b) Calcule la mediana.

c) ¿Cuál reportaría?

Problema 3

Memoria

Una matriz tiene $5 \cdot 10^8$ entradas.

a) GB en float64 (8 B).

b) GB en float32 (4 B).

c) ¿Qué pierde con float32?

→ **Pista:** para la mediana, ordene los cinco valores y tome el del medio.

Solución paso a paso

La conclusión depende de cómo se resume la medición.

1 Aceleración

12 s → 0,03 s

Razón = 12 / 0,03 = 400

Medir: calentar, repetir y usar la mediana

Resultado: 400 veces más rápido

2 Media y mediana

8,0 | 8,2 | 8,4 | 8,2 | 19,3

Ordenados: 8,0 · 8,2 · 8,2 · 8,4 · 19,3

Media = 52,1 / 5 = 10,42

Mediana = 8,2

Resultado: reportar 8,2 ms

3 Memoria

float64: 4 GB | **float32: 2 GB**

$5 \cdot 10^8 \cdot 8$ B = 4 GB

$5 \cdot 10^8 \cdot 4$ B = 2 GB

float32 pierde cifras: $\varepsilon \approx 1{,}2 \cdot 10^{-7}$

Resultado: float32 usa la mitad

→ **Verifique:** con un valor atípico, la media y la mediana pueden diferir mucho.

LABORATORIO · PYTHON

Laboratorio: compruébelo con código

Ejecute en Colab y compare con lo calculado a mano.

1 · Rango, espacio nulo y condición

```
import numpy as np
from scipy import linalg, sparse

A = np.array([[1,2,3],[2,4,6],[0,1,1]])
np.linalg.matrix_rank(A)        # 2
linalg.null_space(A).round(3).T
# [[ 0.577  0.577 -0.577]]

X = np.array([[1,1],[1,1.0001]])
np.linalg.cond(X)               # 40002.0
```

2 · Benchmark: densa vs CSR

```
import time, statistics
def bench(f, r=5):
    f()                         # calentar
    t = []
    for _ in range(r):
        s = time.perf_counter(); f()
        t.append(time.perf_counter() - s)
    return statistics.median(t) * 1e3
M = sparse.random(2000, 2000, density=0.01, format="csr")
D = M.toarray(); x = np.random.rand(2000)
bench(lambda: D @ x)    # ≈ 1,5 ms
bench(lambda: M @ x)    # ≈ 0,05 ms
```

1. Verifique $A\cdot(0{,}577;\ 0{,}577;\ -0{,}577) \approx 0$
2. Cambie 1.0001 por 1.000001: ¿cómo cambia cond?
3. Suba la densidad a 10 %: ¿sigue ganando CSR?

→ **Nota:** los tiempos dependen de su equipo; la proporción entre densa y dispersa es lo que debe comparar.

CIERRE

Ocho ideas para llevarse

Cuatro sobre lo que hace una matriz y cuatro sobre lo que cuesta calcularla.

1. Transformar

Las columnas de A son las imágenes de e_1 y e_2.

2. Col(A) y Nul(A)

Lo que A produce y lo que A anula.

3. Rango

Columnas independientes; rango + nulidad = n.

4. Colinealidad

Columnas parecidas: X^TX casi singular, β inestable.

5. Vectorizar

Operar arreglos completos, sin ciclos for.

6. Dispersas

CSR guarda solo los no ceros.

7. Complejidad

$2n$, $2n^2$, $2n^3$: cuente antes de correr.

8. Benchmarking

Calentar, repetir, mediana; float32 o float64.

→ **Próxima clase:** autovalores y autovectores: las direcciones que una matriz no gira.

LEAD UNIVERSITY

Lo esencial de hoy

1. Una matriz es una función: sus columnas dicen qué hace.
2. El rango dice cuánta información se conserva.
3. El costo se estima contando operaciones y se confirma midiendo.

Próxima clase: autovalores y autovectores